**A**

**Minor Project- II Report**

**On**

# BRAIN SIGNAL CLASSIFICATION USING EEG AND MACHINE LEARNING

**Submitted to**

**RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA, BHOPAL (M.P.)**

**In Partial Fulfillment of the award of the degree of**

**BACHELOR OF TECHNOLOGY**

**IN**

**ELECTRONICS & COMMUNICATION**

**By**

Saloni Parihar (0818EC231055)

Siddharth Pal (0818EC231062)

**Under the Guidance of**

Mr. Ankit Muley

**DEPARTMENT OF ELECTRONICS & COMMUNICATION**

**INDORE INSTITUTE OF SCIENCE & TECHNOLOGY, INDORE-453331 (M.P)**

# DECLARATION

We hereby declare that the work presented in this project entitled " **BRAIN SIGNAL CLASSIFICATION USING EEG AND MACHINE LEARNING**" in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Electronics & Communication Engineering is an authentic record of work carried out by us. The matter embodied in this project has not been submitted by us for the award of any degree.

Saloni Parihar (0818EC231055) Date

Siddharth Pal (0818EC231062) 28th Jun 2026

# CERTIFICATE

This is to certify that the project entitled "**BRAIN SIGNAL CLASSIFICATION USING EEG AND MACHINE LEARNING**" submitted to Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal, by **Saloni Parihar (0818EC231055)** and **Siddharth Pal (0818EC231062)** in partial fulfillment of the requirement for the award of the degree, with specialization in Electronics & Telecommunication Engineering. The matter embodied is the actual work done by all the mentioned members, and this work has not been submitted earlier for the award of any other diploma or degree.

**GUIDED BY**

Mr. Ankit Muley

**Project Coordinator**

Dr. Nitin Chauhan

Mr. Ankit Muley

**HOD ECE**

Dr. Ankit Saxena

# APPROVAL CERTIFICATE

This is to certify that the project entitled "**BRAIN SIGNAL CLASSIFICATION USING EEG AND MACHINE LEARNING**" submitted to Rajiv Gandhi Proudyogiki Vishwavidyalaya (RGPV), Bhopal (M.P.) in the Department of Electronics and Communication Engineering by **Saloni Parihar (0818EC231055)** and **Siddharth Pal (0818EC231062)**, in partial fulfillment of the requirement for the award of the degree Of Bachelor of Engineering in Electronics and Communication Engineering.

**Signature of coordinator**

**with Date**

**Signature of internal**

**with Date**

# ACKNOWLEDGEMENT

The success and outcome of this project required a lot of guidance and assistance from many people, and we are extremely fortunate to have got this all along with the completion of our project work. Whatever we have done is only due to such guidance and assistance and we would not forget to thank them.

We owe our profound gratitude to our project coordinators **Dr. Nitin Chauhan and Mr. Ankit Muley**. who took an interest in our project work, all along, till the completion of our project work by providing all the necessary information, constant encouragement, sincere criticism, and a sympathetic attitude. The completion of this dissertation would not have been possible without such guidance and support.

We owe our profound gratitude to our project Guide **Mr. Ankit Muley**, who took a keen interest in our project work and guided us.

We extend our deep gratitude to **Mr. Ankit Saxena** , HOD, Department of Electronics & Communication, for his/her support and suggestions during this project Work.

We respect and thank our Hon'ble Principal **Dr/Prof Keshav Patidar**, for allowing us to do the project work on campus and providing us with all the necessary resources, support, and constant motivation which made us complete the project on time.

We are thankful to and fortunate enough to get constant encouragement and guidance from all teaching staff of the Department of Electronics & Communication which helped us in completing our project work. We would like to extend our sincere regards to all the non-teaching staff of the Department of Electronics &Communication for their timely support.

# TABLE OF CONTENTS

# ABSTRACT

Electroencephalography (EEG) is a non-invasive technique used to measure brain activity and has become an important tool in Brain–Computer Interface (BCI) systems. This project focuses on the classification of EEG brain signals using machine learning techniques and the application of classified signals for device control. EEG data obtained from the PhysioNet dataset are preprocessed to remove noise and artifacts, followed by feature extraction and classification using machine learning algorithms. The trained model identifies different brain activity patterns with high accuracy. To demonstrate the practical application of the system, pre-recorded EEG signals are used to control an LED, simulating a brain-controlled device. The results show that machine learning can effectively classify EEG signals and support the development of intelligent BCI applications for assistive technologies and human–computer interaction.

This project presents a machine learning-based approach for classifying EEG brain signals using data from the PhysioNet dataset. After preprocessing and feature extraction, machine learning models are trained to identify different brain activity patterns. The classified signals are then used to control an LED through prerecorded EEG data, demonstrating a simple Brain–Computer Interface (BCI) application. The results show that machine learning can effectively classify EEG signals and support real-world applications such as assistive technologies, smart device control, and human–computer interaction.

# CHAPTER 1

# INTRODUCTION

**Electroencephalography (EEG)** is a non-invasive technique used to record the electrical activity of the brain through electrodes placed on the scalp. EEG signals provide valuable information about brain functions and are widely used in medical diagnosis, neuroscience research, and **Brain–Computer Interface (BCI)** systems. BCI technology enables direct communication between the human brain and external devices by interpreting neural signals and translating them into actionable commands.

Recent advancements in machine learning have significantly improved the ability to analyze and classify EEG signals. Traditional signal processing methods often struggle with the complex, non-linear, and noisy nature of EEG data. Machine learning algorithms can automatically identify patterns within EEG signals, making them highly suitable for brain signal classification tasks. Accurate classification of EEG signals is essential for applications such as assistive technologies, rehabilitation systems, smart device control, and human–computer interaction.

This project focuses on the classification of EEG brain signals using machine learning techniques and data obtained from the **PhysioNet** dataset. The EEG signals undergo preprocessing and feature extraction before being used to train classification models. To demonstrate the practical application of the developed system, prerecorded EEG signals are utilized to control an LED, representing a simple brain-controlled device. The project highlights the potential of combining EEG signal processing and machine learning to develop efficient and intelligent BCI systems.

## 1.1 Objective

The main objectives of this project are:

- To acquire and analyze EEG data from the PhysioNet dataset.
- To preprocess EEG signals and remove noise and artifacts.
- To extract meaningful features from EEG data for classification.

- To implement machine learning algorithms for brain signal classification.
- To evaluate the performance of the classification model.
- To demonstrate a practical Brain–Computer Interface application by controlling an LED using classified prerecorded EEG signals.

## 1.2 Organization

The report is organized into the following chapters:

- **Chapter 1:** Introduction, objectives, motivation, and report organization.
- **Chapter 2:** Literature review and background study on EEG signal processing, Brain–Computer Interfaces, and machine learning techniques.
- **Chapter 3:** Methodology, including dataset description, preprocessing, feature extraction, and classification approaches.
- **Chapter 4:** System implementation and experimental setup.
- **Chapter 5:** Results, performance evaluation, and discussion.
- **Chapter 6:** Conclusion and future scope of the project.

## 1.3 Motivation

The development of an EEG-based brain signal classification system is motivated by the growing demand for intelligent Brain–Computer Interface (BCI) applications that enable direct communication between the human brain and external devices. The use of machine learning techniques and EEG data from the PhysioNet dataset is driven by the following factors:

### 1.3.1 Advancement in Brain–Computer Interface Technology

Brain–Computer Interface systems provide a direct pathway between the human brain and machines without requiring physical movement. EEG signals contain valuable information about brain activities and can be used to generate control commands. Developing an EEG classification system contributes to the advancement of BCI technology and demonstrates how neural signals can be utilized for practical device control applications.

### 1.3.2 Intelligent Signal Classification Using Machine Learning

EEG signals are highly complex, non-linear, and often affected by noise and artifacts. Traditional analysis methods face challenges in accurately identifying meaningful patterns from such data. Machine learning algorithms offer powerful tools for feature extraction and classification, enabling the system to recognize different brain activity patterns with improved accuracy and reliability. This motivates the adoption of machine learning techniques for efficient EEG signal processing.

### 1.3.3 Practical Application and Assistive Technology Development

One of the major motivations of this project is to demonstrate the real-world applicability of EEG-based systems. By using prerecorded EEG signals to control an LED, the project simulates a brain-controlled device and showcases the potential of BCI technology in assistive applications. Such systems can help individuals with physical disabilities interact with their environment, control smart devices, and improve their quality of life.

# CHAPTER 2

# LITERATURE SURVEY

1. **Ramírez-Arias, Francisco Javier, et al. "Evaluation of machine learning algorithms for classification of EEG signals." Technologies 10.4 (2022): 79.**

This paper reviews machine learning techniques such as SVM, LDA, KNN, and ANN for EEG signal classification in Brain-Computer Interface (BCI) systems. The study shows that machine learning improves classification accuracy and supports applications in rehabilitation and robotics. However, it mainly focuses on traditional algorithms and lacks real-time implementation.

2. **Ekinci, Ekin, Zeynep Garip, and Kasım Serbest. "Electromyography based hand movement classification and feature extraction using machine learning algorithms." Politeknik Dergisi 26.4 (2023): 1621-1633.**

The authors present an EEG-based hand movement detection system using Artificial Neural Networks. EEG signals are processed and used to train the ANN model for movement classification. The system achieves good accuracy but requires large training datasets and does not include real-time hardware control.

3. **Al-jumaili, Saif, et al. "Investigation of epileptic seizure signatures classification in EEG using supervised machine learning algorithms." *Traitement du Signal* 40.1 (2023): 43.**

This study focuses on classifying epileptic seizure patterns from EEG signals using supervised machine learning techniques. The results show effective seizure detection and classification. However, the work is mainly intended for medical diagnosis rather than BCI applications.

4. **Shi, Jianwei, et al. "EPAT: a user-friendly MATLAB toolbox for EEG/ERP data processing and analysis." *Frontiers in Neuroinformatics* 18 (2024): 1384250.**

This paper uses deep learning models such as CNNs and RNNs for EEG signal classification. The approach improves classification performance by automatically extracting features from EEG data. However, it requires high computational resources and large datasets.

5. **Yu, Hui, et al. "Technical system of electroencephalography-based brain–computer interface: Advances, applications, and challenges."** ***Neural Regeneration Research*** **21.9 (2026): 3885-3907.**

This review discusses recent developments in EEG-based BCI systems, including signal processing methods, classification techniques, and applications. It highlights challenges such as noise, user variability, and real-time processing limitations. However, it does not propose a specific implementation framework.

# CHAPTER 3

# METHODOLOGY

## 3.1 Software Description

The proposed system is designed to classify EEG signals corresponding to different mental tasks and control an LED through an ESP32 microcontroller. The complete methodology consists of the following stages:

### 3.1.1 EEG Data Acquisition

The EEG signals used in this project are obtained from the PhysioNet EEG dataset. The dataset contains brain activity recordings collected from multiple subjects performing different mental tasks such as left-hand movement, right-hand movement, relaxation, and motor imagery. These EEG recordings serve as the input data for the proposed system.

### 3.1.2 Signal Preprocessing

Raw EEG signals often contain unwanted noise, artifacts, and interference caused by muscle movements, eye blinks, and external electrical sources. To improve signal quality, preprocessing techniques such as band-pass filtering, notch filtering, and normalization are applied. This step removes unwanted components and enhances the quality of the EEG signals for further analysis.

### 3.1.3 Feature Extraction

After preprocessing, important features are extracted from the EEG signals to represent the underlying brain activity effectively. Both time-domain and frequency-domain features are considered. Time-domain features include mean, variance, standard deviation, and root mean square (RMS), while frequency-domain features include power spectral density and frequency band power. These features form the input vector for the classification model.

### 3.1.4 Dataset Preparation

The extracted feature vectors are labeled according to their corresponding mental tasks. The complete dataset is then divided into training and testing sets, typically using an 80:20 ratio. The training set is used to train the model, while the testing set is used to evaluate its performance and classification accuracy.

### 3.1.5 ANN Model Training

An Artificial Neural Network (ANN) is developed and trained using the prepared feature dataset. During training, the ANN learns the relationship between the extracted EEG features and their corresponding mental task labels. Various network parameters such as the number of hidden neurons, learning rate, and training epochs are optimized to achieve better classification performance.

### 3.1.6 EEG Signal Classification

Once the ANN is trained, it is used to classify unseen EEG signals. The trained model analyzes the input features and predicts the user's intended mental task based on the learned patterns. The output of the classifier corresponds to different brain activity classes such as relaxation, left-hand movement, right-hand movement, or motor imagery.

### 3.1.7 Command Generation

The classification results obtained from the ANN are converted into predefined command codes. Each mental task is assigned a unique command that can be interpreted by the ESP32 microcontroller. This conversion enables the classified brain activity to be translated into a practical control action.

### 3.1.8 ESP32 Communication

The generated command is transmitted from MATLAB to the ESP32 microcontroller through

serial communication. MATLAB acts as the transmitter, while the ESP32 functions as the receiver. The communication process ensures reliable transfer of classification results to the hardware system.

### 3.1.9 LED Control

After receiving the command, the ESP32 processes the received data and controls the LED connected to its GPIO pins. Different commands correspond to different LED actions such as turning the LED ON, turning it OFF, or generating specific blinking patterns.

### 3.1.10 Output Indication

The LED serves as a visual indicator of the classified brain activity. Each mental task is associated with a unique LED state, allowing users to observe the classification result directly. This demonstrates the successful implementation of a Brain-Computer Interface (BCI) system that translates EEG signals into real-time hardware control actions.

## 3.2 Hardware Description

The hardware section of the proposed Brain–Computer Interface (BCI) system consists of an ESP32 microcontroller and an LED output device. Although EEG data is obtained from the PhysioNet dataset and processed in MATLAB, the classified results are transmitted to the ESP32, which performs the final hardware control operation.

#### 3.2.1 ESP32 Microcontroller

The ESP32 is a low-cost, high-performance microcontroller equipped with built-in Wi-Fi and Bluetooth capabilities. It acts as the central hardware unit of the system. The ESP32 receives classification commands from MATLAB through serial communication and processes them to control the connected output device. Due to its high processing speed, low power consumption, and communication capabilities, the ESP32 is widely used in IoT and embedded system applications.

### 3.2.2 Light Emitting Diode (LED)

An LED is used as the output indicator in the proposed system. The ESP32 controls the LED based on the command generated from the classified EEG signal. Different brain activity classes can be represented through different LED states such as ON, OFF, slow blinking, or fast blinking. The LED provides a simple visual demonstration of the successful implementation of the Brain–Computer Interface.

### 3.2.3 USB Communication Interface

A USB cable is used to establish serial communication between MATLAB and the ESP32 microcontroller. The classification results generated by the Artificial Neural Network (ANN) are transmitted through the serial port, enabling real-time communication between the software and hardware sections of the system.

### 3.2.4 Personal Computer (PC/Laptop)

A computer system is used to execute MATLAB programs, perform EEG signal preprocessing, extract features, train the ANN model, classify EEG signals, and generate control commands. The computer serves as the processing unit of the entire system.

---

**Hardware Configuration**

| Component | Purpose |
|---|---|
| ESP32 Microcontroller | Receives commands and controls output devices |
| LED | Visual indication of classified brain activity |
| USB Cable | Serial communication between MATLAB and ESP32 |
| PC/Laptop | EEG processing, ANN training, and classification |

## 3.3 Software

- **MATLAB**

MATLAB is the primary software platform used in this project for EEG signal processing, feature extraction, Artificial Neural Network (ANN) training, and classification. It provides a powerful environment for data analysis, machine learning, and visualization of EEG signals.

**• Neural Network Toolbox**

The Neural Network Toolbox is used to design, train, validate, and test the Artificial Neural Network model. It provides built-in functions for implementing ANN architectures and optimizing classification performance.

**• Signal Processing Toolbox**

The Signal Processing Toolbox is used for preprocessing EEG signals. It provides filtering, normalization, frequency analysis, and noise removal techniques that improve the quality of the EEG data before feature extraction.

**• Arduino IDE**

Arduino IDE is an open-source development environment used to write, compile, and upload firmware to the ESP32 microcontroller. It enables communication between the classification system and the hardware output device.

**• ESP32 Board Package**

The ESP32 Board Package allows the Arduino IDE to program and configure the ESP32 microcontroller. It provides the necessary drivers and libraries required for serial communication and GPIO control.

**• Serial Communication Library**

The serial communication library is used to establish communication between MATLAB and the ESP32 microcontroller. It enables the transmission of classification commands from the ANN model to the hardware system in real time.

## 3.4 Block Diagram

The proposed system is an EEG-Based Hand Movement Detection System using Artificial Neural Network (ANN). The system acquires EEG signals from a dataset and processes them through multiple stages to identify the user's intended hand movement. Based on the prediction result, an LED indicator is controlled.

**Working Principle**

1. EEG Dataset Acquisition
    - EEG signals corresponding to left-hand and right-hand movements are collected from the dataset.
    - These signals serve as the input to the system.
2. Data Preprocessing
    - Raw EEG signals often contain noise and unwanted artifacts.
    - Preprocessing is performed to clean the data and prepare it for further analysis.
3. Signal Filtration
    - A bandpass filter (8–30 Hz) is applied to remove irrelevant frequencies and retain useful brain activity related to motor movements.
4. Feature Extraction
    - Important statistical features such as Mean, Standard Deviation, Variance, Maximum, and Minimum values are extracted from the filtered EEG signals.
    - These features represent the characteristics of brain activity.
5. ANN-Based Classification
    - The extracted features are provided as input to an Artificial Neural Network (ANN).
    - The ANN is trained using labeled EEG data representing left-hand and right-hand movements.
6. Model Training
    - During training, the ANN learns patterns associated with different hand movements.
    - The trained model achieved an accuracy of approximately 87%.
7. Model Testing
    - The trained model is tested using unseen EEG samples to evaluate its performance and prediction capability.

8. Output Generation
    - The ANN predicts the intended hand movement based on the EEG signal.
    - The prediction result is used to generate an output command.
9. LED Control
    - If the predicted movement matches the predefined condition, the LED is turned ON.
    - Otherwise, the LED remains OFF.
    - This demonstrates the practical implementation of a Brain-Computer Interface (BCI) system.

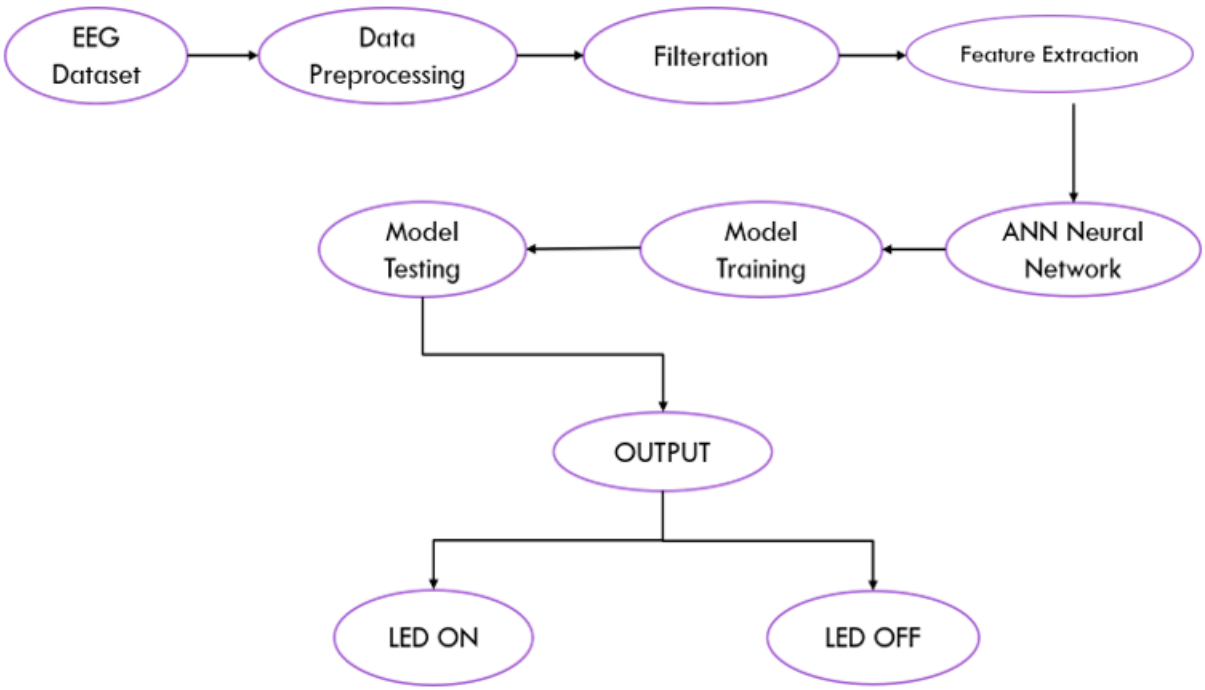


**Fig. 1. Block diagram of proposed method**

# CHAPTER 4
# RESULT

The proposed EEG-based Brain–Computer Interface system was successfully implemented using MATLAB for EEG signal classification and an ESP32 Nano for hardware control. EEG signals obtained from the PhysioNet dataset were preprocessed, filtered, and transformed into feature vectors before being used for ANN training and testing.

The Artificial Neural Network effectively learned the patterns present in the EEG data and classified the signals into predefined classes. The trained model demonstrated satisfactory performance in recognizing brain activity patterns and generating corresponding output commands. The classification results were transmitted to the ESP32 Nano microcontroller, which successfully controlled the connected LEDs according to the predicted brain activity class.

The hardware implementation verified the successful integration of machine learning and embedded systems. When a specific EEG class was detected, the corresponding LED was activated, demonstrating the conversion of brain signal classifications into physical actions. This confirms the feasibility of using ANN-based EEG classification for Brain–Computer Interface applications.

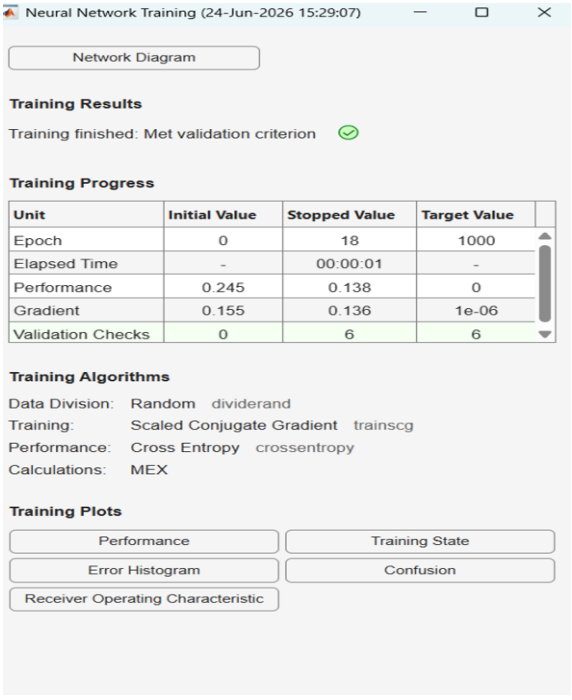


Fig. 2. Neural Network Training

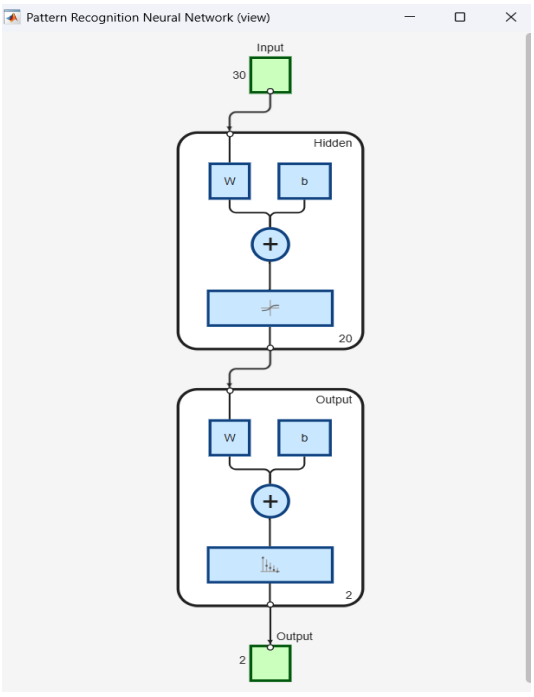


Fig. 3. Pattern Recognization

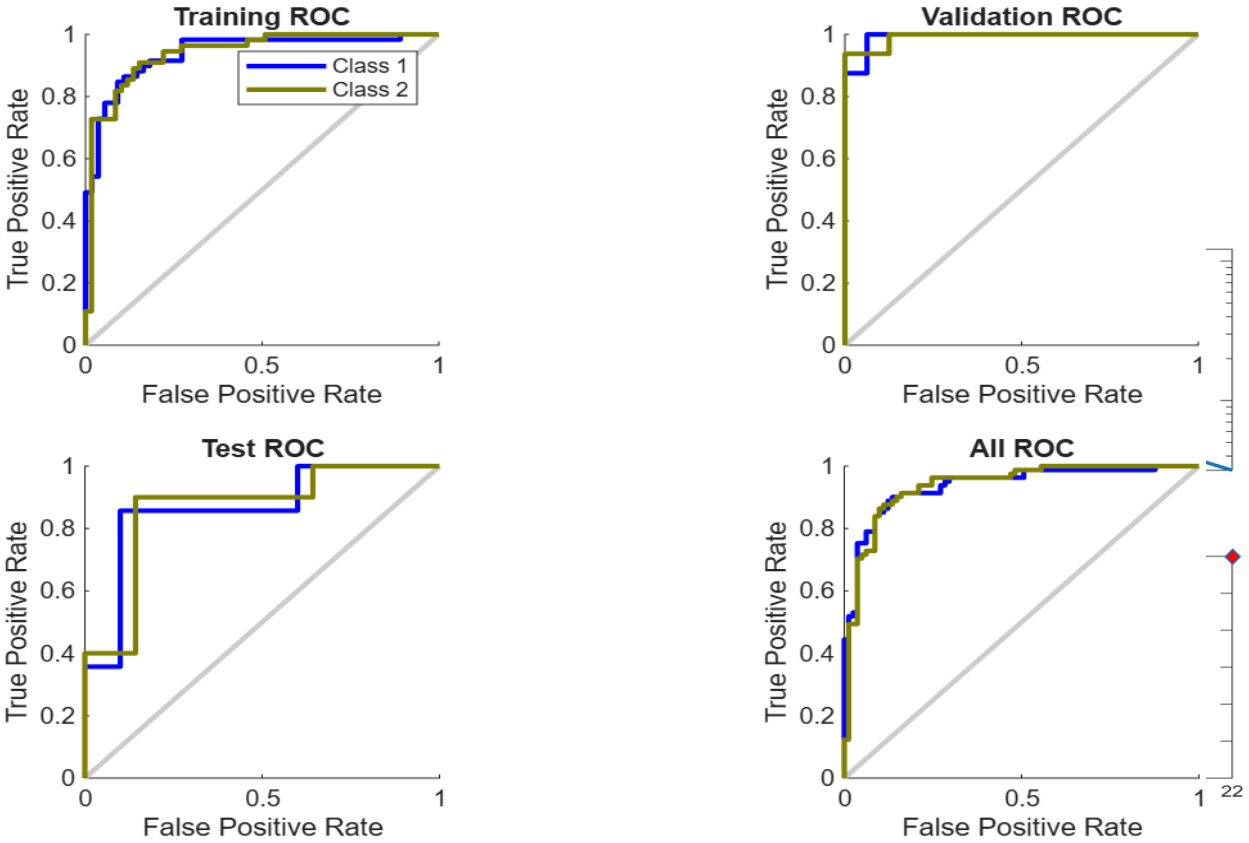


Fig. 4. ROC Plot

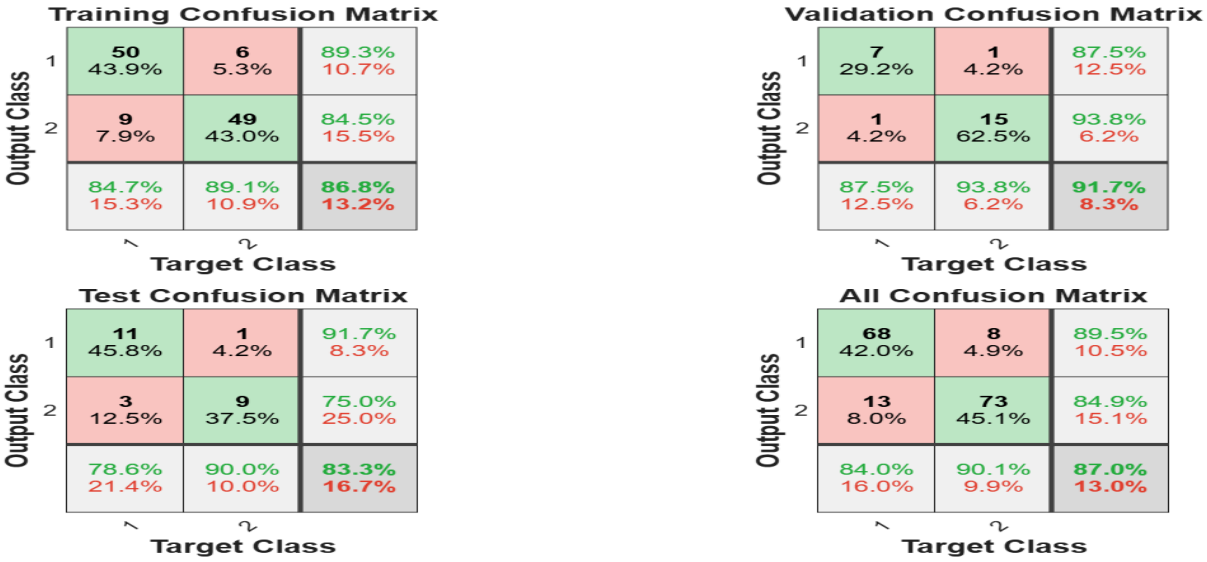


Fig. 5. Confusion Matrix

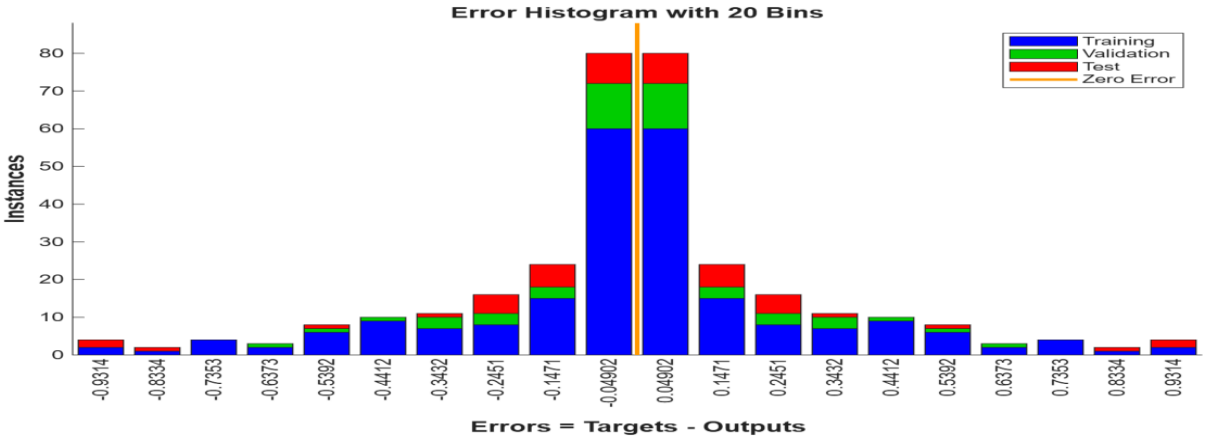


Fig. 6. Histogram

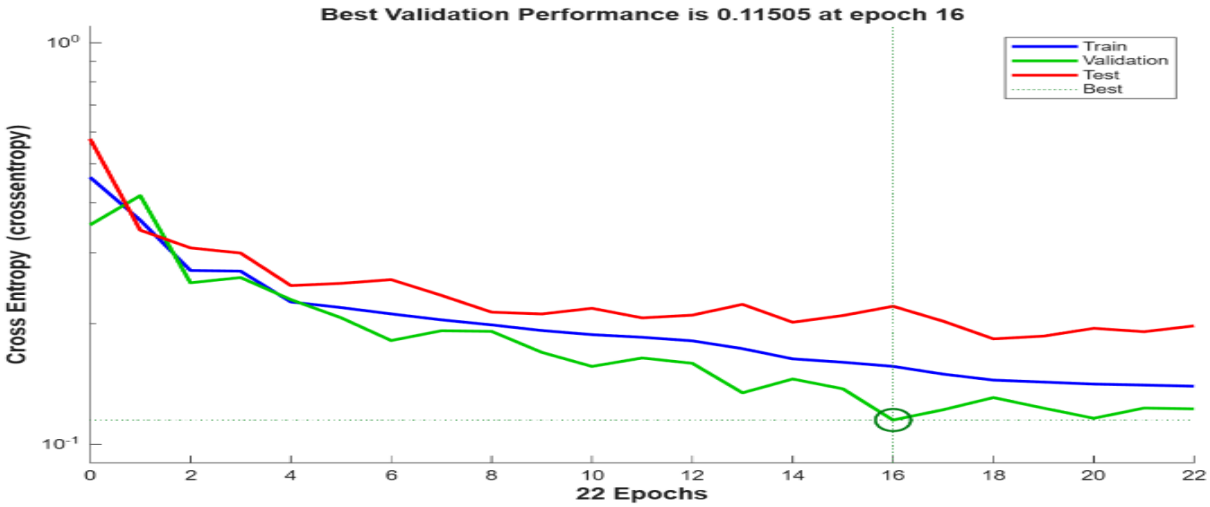


Fig. 7. Performance

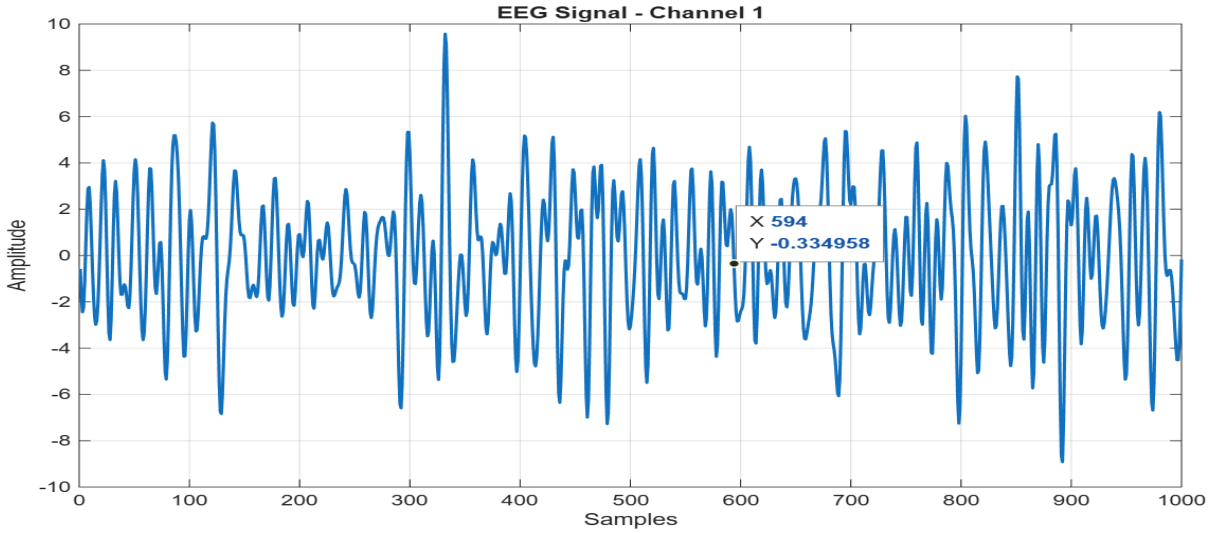


Fig. 8. EEG Signal

# CHAPTER 5
# CONCLUSION & FUTURE SCOPE

The proposed Brain Control Interface system uses EEG signals and an Artificial Neural Network (ANN) to classify brain activity and generate control commands. These commands are sent to an ESP32 microcontroller, which controls an LED based on the detected brain signals. The project demonstrates the potential of EEG-based Brain–Computer Interfaces for enabling direct communication between the human brain and external devices.

## Future Scope

### 1. Real-Time EEG Signal Acquisition

The proposed system currently uses pre-recorded EEG datasets for training and testing. In the future, it can be integrated with real-time EEG acquisition devices such as OpenBCI or Emotiv headsets.

### 2. Integration with Robotic Arms and Assistive Devices

The classified EEG signals can be utilized to control robotic arms, wheelchairs, prosthetic limbs, and other assistive technologies.

### 3. Implementation of Advanced Deep Learning Models

The Artificial Neural Network (ANN) can be replaced or enhanced with advanced deep learning architectures such as Convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTM) networks.

### 4. Multi-Device Control System

Future versions of the system can be expanded to control multiple electronic devices simultaneously. Different EEG classifications can be mapped to various home automation systems, robots, or IoT devices, creating a more versatile brain-controlled environment.

### 5. Smart Home Automation

The proposed system can be integrated with smart home technologies to control lights, fans, doors, appliances, and security systems using brain signals. This application can provide greater convenience and accessibility for users.

**6. Healthcare and Rehabilitation Applications**

The system can be used in rehabilitation programs to assist stroke patients and individuals with neurological disorders. EEG-based monitoring and control can support therapy sessions and help track patient recovery progress.

**7. Wireless Communication and IoT Integration**

Future implementations can utilize wireless communication protocols such as Wi-Fi, Bluetooth, or MQTT for seamless communication between the EEG processing unit and connected devices. This will improve portability and scalability.

**8. Enhanced User Interface and Feedback System**

A graphical user interface (GUI) can be developed to display EEG signals, classification results, device status, and performance metrics in real time. Visual, audio, or haptic feedback can also be added to improve user interaction.

**9. Increased Number of Mental Commands**

The current system can be extended to recognize a larger set of mental tasks and commands. This will allow users to control more devices and perform more complex operations through brain activity.

**10. Development of a Complete Brain–Computer Interface Platform**

The project can evolve into a comprehensive Brain–Computer Interface platform capable of real-time signal acquisition, intelligent classification, wireless communication, and control of multiple external devices for healthcare, industrial, and consumer applications.

# REFERENCES

1. Ramírez-Arias, Francisco Javier, et al. "Evaluation of machine learning algorithms for classification of EEG signals." *Technologies* 10.4 (2022): 79.
2. Ekinci, Ekin, Zeynep Garip, and Kasım Serbest. "Electromyography based hand movement classification and feature extraction using machine learning algorithms." *Politeknik Dergisi* 26.4 (2023): 1621-1633.
3. Al-jumaili, Saif, et al. "Investigation of epileptic seizure signatures classification in EEG using supervised machine learning algorithms." *Traitement du Signal* 40.1 (2023): 43.
4. Shi, Jianwei, et al. "EPAT: a user-friendly MATLAB toolbox for EEG/ERP data processing and analysis." *Frontiers in Neuroinformatics* 18 (2024): 1384250.
5. Yu, Hui, et al. "Technical system of electroencephalography-based brain–computer interface: Advances, applications, and challenges." *Neural Regeneration Research* 21.9 (2026): 3885-3907.

# SOURCE CODE

**Main Code**

```
clc;
clear all;
close all;

%% Ans1
load('./Dataset/SubB_6chan_2LR.mat');
EEGDATA=squeeze(EEGDATA(1,:,:));
Ans1=NN(EEGDATA,LABELS');

%% Ans2
load('./Dataset/SubC_6chan_2LR_s5.mat');
EEGDATA=squeeze(EEGDATA(1,:,:));
Ans2=NN(EEGDATA,LABELS');

%% Ans3
load('./Dataset/SubC_6chan_3LRF_s1.mat');
EEGDATA=squeeze(EEGDATA(1,:,:));
Ans3=NN(EEGDATA,LABELS');

%% Ans4
load('./Dataset/SubC_6chan_3LRF_s2.mat');
EEGDATA=squeeze(EEGDATA(1,:,:));
Ans4=NN(EEGDATA,LABELS');


%% Ans5
load('./Dataset/SubC_6chan_3LRF_s3.mat');
EEGDATA=squeeze(EEGDATA(1,:,:));
Ans5=NN(EEGDATA,LABELS');
```

```
%% Ans6
load('./Dataset/SubD_5chan_2LR.mat');
EEGDATA=squeeze(EEGDATA(1,:,:));
Ans6=NN(EEGDATA,LABELS');

%% Ans7
load('./Dataset/SubE_5chan_2LR.mat');
EEGDATA=squeeze(EEGDATA(1,:,:));
Ans7=NN(EEGDATA,LABELS');

%% Ans8
load('./Dataset/SubF_6chan_2LR.mat');
EEGDATA=squeeze(EEGDATA(1,:,:));
Ans8=NN(EEGDATA,LABELS');

%% Ans9
load('./Dataset/SubG_6chan_2LR.mat');
EEGDATA=squeeze(EEGDATA(1,:,:));
Ans9=NN(EEGDATA,LABELS');


%% Ans10
load('./Dataset/SubH_6chan_2LR.mat');
EEGDATA=squeeze(EEGDATA(1,:,:));
Ans10=NN(EEGDATA,LABELS');
```

**CreateNet.m**

```
function Ans=CreateNet(Input,Target)
net = newff(Input,Target,[15]);
% net = newff(Input,Target,[15]);
net.trainParam.min_grad = 0.000001;
%net.trainParam.epochs = 500;
```

```
net.divideParam.trainRatio = 75/100;
net.divideParam.valRatio = 10/100;
net.divideParam.testRatio = 15/100;
net.trainParam.max_fail = 15;
net.trainParam.showWindow=0;
% net.layers{1}.transferFcn = 'logsig';
Ans=net;
End
```

**GetMSEs.m**

```
function [TrainMSE, TestMSE, TrainErrorRate, TestErrorRate] = GetMSEs(net, TrainIns,
TrainGoals, TestIns, TestGoals)
TestOuts = net(TestIns);
TestMSE = mse(net, TestGoals, TestOuts);
[m1,n1] = max(TestOuts);
[m2,n2] = max(TestGoals);
errors = (n1~=n2);
TestErrorsCount = sum(sum(errors));
TestErrorRate = round(10000*(TestErrorsCount / size(TestIns,2)))/100;
TrainOuts = net(TrainIns);
TrainMSE = mse(net, TrainGoals, TrainOuts);
[m1,n1] = max(TrainOuts);
[m2,n2] = max(TrainGoals);
errors = (n1~=n2);
TrainErrorsCount = sum(sum(errors));
TrainErrorRate = round(10000*(TrainErrorsCount / size(TrainIns,2)))/100;
End
```

**NN.m**

```
function Ans=NN(Inputs,Targets)
%% Supervised
```

```
  %1
 [TrainMSE1, TestMSE1, TrainErrorRate1, TestErrorRate1]  =....
   Supervised(Inputs,Targets,1);
  %2
 [TrainMSE2, TestMSE2, TrainErrorRate2, TestErrorRate2]  =....
   Supervised(Inputs,Targets,2);
  %3
 [TrainMSE3, TestMSE3, TrainErrorRate3, TestErrorRate3]  =....
    Supervised(Inputs,Targets,3);
  %4
 [TrainMSE4, TestMSE4, TrainErrorRate4, TestErrorRate4]  =....
   Supervised(Inputs,Targets,4);
%% SemiSupervised
[TrainMSE5, TestMSE5, TrainErrorRate5, TestErrorRate5]  =....
   SemiSupervised(Inputs,Targets);

%% return
Ans=cell(1,4);
Ans(1)={[TrainMSE1, TestMSE1, TrainErrorRate1, TestErrorRate1]};
Ans(2)={[TrainMSE2, TestMSE2, TrainErrorRate2, TestErrorRate2]};
Ans(3)={[TrainMSE3, TestMSE3, TrainErrorRate3, TestErrorRate3]};
Ans(4)={[TrainMSE4, TestMSE4, TrainErrorRate4, TestErrorRate4]};
Ans(5)={[TrainMSE5, TestMSE5, TrainErrorRate5, TestErrorRate5]};
End
```

**SemiSupervised.m**

```
function [TrainMSE, TestMSE, TrainErrorRate, TestErrorRate]  =
SemiSupervised(Inputs,Targets)
n=size(Inputs,2)/10;
TrainInputs=Inputs(:,1:floor(n));
TrainTargets=Targets(:,1:floor(n));
TestInputs=Inputs(:,floor(n*8)+1:end);
```

```
TestTargets=Targets(:,floor(n*8)+1:end);
%% Training
[net, tr] = train(CreateNet(TrainInputs, TrainTargets), TrainInputs, TrainTargets);
TrainInputs=[TrainInputs,Inputs(:,floor(n)+1:floor(2*n))];
TrainTargets=[TrainTargets,net(Inputs(:,floor(n)+1:floor(2*n)))];
[net, tr] = train(CreateNet(TrainInputs, TrainTargets), TrainInputs, TrainTargets);
TrainInputs=[TrainInputs,Inputs(:,floor(2*n)+1:floor(4*n))];
TrainTargets=[TrainTargets,net(Inputs(:,floor(2*n)+1:floor(4*n)))];
[net, tr] = train(CreateNet(TrainInputs, TrainTargets), TrainInputs, TrainTargets);
TrainInputs=[TrainInputs,Inputs(:,floor(4*n)+1:floor(8*n))];
TrainTargets=[TrainTargets,net(Inputs(:,floor(4*n)+1:floor(8*n)))];
[net, tr] = train(CreateNet(TrainInputs, TrainTargets), TrainInputs, TrainTargets);
EpochCounts = tr.num_epochs;
%%
[TrainMSE, TestMSE, TrainErrorRate, TestErrorRate] = GetMSEs(net, TrainInputs,
TrainTargets, TestInputs, TestTargets);
End
```

**Supervised.m**

```
function [TrainMSE, TestMSE, TrainErrorRate, TestErrorRate]  =
Supervised(Inputs,Targets,number)
%% select train and test
n=size(Inputs,2);
NTest=number:4:n;
NTrain=1:n;
n=size(NTest,2);
for i=1:n
NTrain=NTrain(NTrain~=NTest(i));
end
TrainInputs=Inputs(:,NTrain);
TrainTargets=Targets(:,NTrain);
TestInputs=Inputs(:,NTest);
```

```
TestTargets=Targets(:,NTest);
%%
[net, tr] = train(CreateNet(TrainInputs,TrainTargets), TrainInputs, TrainTargets);
save('EEG_ANN_Model.mat','net','tr');
EpochCounts = tr.num_epochs;
[TrainMSE, TestMSE, TrainErrorRate, TestErrorRate]  = GetMSEs(net, TrainInputs,
TrainTargets, TestInputs, TestTargets);
end
```